Smart Env Monitor – User Guide

This guide is written for the municipality's users (both technical and non-technical) who will use Smart Env Monitor to monitor bathing water quality and related environmental data.

The guide describes:

- How the system is built
- How to start the system (Docker + simulator)
- How to use the dashboard
- How roles and permissions work
- How alert rules and monitoring work
- Common errors and how to resolve them

In short: The system simulates a real IoT flow: sensors → API → database → API → dashboard.

---

1. System Overview

The system consists of the following main parts:

- **Sensor simulator** (Python)
  - Reads a dataset of historical measurements from automatic sensors in Chicago.
  - Continuously sends measurements to the backend via HTTP (POST /api/data/) using a secret API key.
- **Backend / API** (Django + Django REST Framework)
  - Receives measurement data and stores it in the database.
  - Exposes data via API endpoints for the dashboard and other potential systems.
  - Validates API keys, enforces rate limiting, and handles alert rules.
- **Database** (PostgreSQL)
  - Stores all sensor readings, alert rules, alert events, and incident reports.
- **Dashboard / Web interface** (Django Templates)
  - Displays current status, historical graphs, sensor health, alert rules, and incidents.
  - Provides different views based on the user's role (Admin, Manager, Analyst, Viewer).
- **Monitoring**
  - Health endpoint at /health/ for technical monitoring.
  - check_sensor_health command that checks whether sensors have stopped sending data.

---

2. Starting the System (Production with Docker)

This is the recommended method for operations and demonstration.

2.1. Prepare .env

1. Copy the file **.env.example to .env** in the project root:
   - On Windows: copy in Explorer or using e.g. copy .env.example .env
2. Open .env and fill in at least:

env

```
SECRET_KEY=<secret_django_key>
```

```
DEVICE_API_KEY=<secret_device_api_key>
DATABASE_URL=postgresql://smartenv:smartenv@db:5432/smart_env_monitor
PROXY_HTTP_PORT=18080
PROXY_HTTPS_PORT=18443
SENSOR_STALE_MINUTES=30
SENSOR_HEALTH_EMAIL_TO=
```

Recommended way to generate secure keys:

bash
```
python -c "import secrets; print('SECRET_KEY=' + secrets.token_urlsafe(48));
print('DEVICE_API_KEY=' + secrets.token_urlsafe(32))"
```

Copy the generated values into .env.

2.2. Start the Docker stack

1. Start **Docker Desktop**.
2. Open a terminal in the project folder:

bash
```
cd c:\Jensen\LIA2\IoT_project\smart-env-monitor
docker compose up -d -build
```

3. Verify that the containers are running:

bash
```
docker compose ps
```

You should see at least:

- db (PostgreSQL)
- web (Django/Gunicorn)
- proxy (Caddy) on ports 18080/18443

2.3. Create an admin user

1. Create an administrator:

bash
```
docker compose exec web python manage.py createsuperuser
```

2. Enter:
   - Username (e.g., admin)
   - Email address (must be valid, e.g., admin@kommun.se)
   - Password (will not be shown; enter twice)
3. Log in to the admin interface at:
   - http://localhost:18080/admin/

2.4. Start the sensor simulator

1. Open a **new** terminal in the project root.
2. Ensure that .env contains:

env
```
BACKEND_URL=http://localhost:18080/api/data/
SENSOR_ID=env-sensor-01
SENSOR_INTERVAL=10
DATASET_PATH=sensor/dataset/environment_data.csv
DEVICE_API_KEY=<same_key_as_in_BACKEND>
```

3. Start the simulator:

bash
```
python sensor/simulator.py
```

4. The simulator will output e.g.:

Starting simulator. Target: **http://localhost:18080/api/data/**

See your live graphs here: **http://localhost:18080/dashboard/**

Reading from: **sensor/dataset/environment_data.csv**
Sent data for Ohio Street Beach: Temp=...

---

3. Dashboard – How to Read Data

3.1. Open the dashboard

Open your web browser and go to:

- **http://localhost:18080/dashboard/**

The dashboard can be viewed without logging in (public view). Login is required for certain tools and report views.

3.2. Current values

At the top, you see the **current value** for:

- Water temperature (°C)
- Turbidity (NTU)
- Wave height (m)
- Battery level (%)

The status box (OK/Warning) summarizes the quality assessment.

3.3. Historical graphs

In the middle of the page, graphs for the last 24 hours (or a custom interval) are shown:

- X-axis: time
- Y-axes: temperature, turbidity, wave height, etc.

You can:

- Select an interval (From / To) and click **Show interval**.
- Switch to live view (24h) using the **Live (24h)** button.

3.4. Sensor status (Stale/OK)

Further down, there is a **Sensor Status** table:

- Each row represents a sensor (e.g., a beach).
- Columns:
    - **Sensor** – sensor ID or beach name.
    - **Last received** – the most recent time data was received.
    - **Status** – OK or Stale.
    - **Age** – minutes since the last measurement.

A sensor becomes **Stale** if it has not reported within SENSOR_STALE_MINUTES (default 30 minutes). This gives a quick overview of whether any sensor has “died” or lost connection.

---

4. Roles and Permissions (RBAC)

The system uses Django groups to distinguish roles. The most important roles are:

- **Admin**
  - Full access.
  - Can use Django Admin (/admin/), create users, modify groups and alert rules.
- **Manager**
  - Can see manager-specific report views, incident lists, and export data.
  - Protected views:
    - /dashboard/manager/
    - /dashboard/reports/
- **Analyst**
  - Has access to analysis views (e.g., /dashboard/analyst/) for deeper data analysis.
- **Viewer**
  - Can see the dashboard and all visualised data (read-only).

Technically:

- Group checks are performed both in views and templates, using e.g. user|has_group:"Manager" and @user_passes_test(...).

4.1. Assign roles

1. Log in as Admin at /admin/.
2. Go to **Users**.
3. Open a user.
4. Under **Groups**, add e.g.:
   - Admin
   - Manager
   - Analyst
   - Viewer
5. Save.

---

5. Alert Rules

Alert rules are used to automatically monitor sensor data and send email when certain conditions are met, for example:

- "Turbidity > 5 NTU on a specific beach"
- "Quality = Poor"

5.1. Where are alert rules located?

Alert rules and alert events are found in the admin interface at:

- /admin/api/alertrule/
- /admin/api/alertevent/

They are also summarised at the bottom of the dashboard:

- **Active alert rules**
- **Recent alert events**

5.2. Create a new alert rule (example)

1. Log in as Admin.

2. Go to /admin/api/alertrule/ and click **Add Alert Rule**.
3. Fill in e.g.:
    - **Name:** High turbidity – Ohio Street Beach
    - **Enabled:** checked
    - **Sensor id:** leave blank to apply to all sensors, or set e.g. Ohio Street Beach.
    - **Metric:** Turbidity
    - **Operator:** > (gt)
    - **Threshold number:** e.g. 5.0
    - **Recipient email:** e.g. environment@kommun.se
    - **Cooldown minutes:** e.g. 30 (at least 30 minutes between alerts from the same rule)
4. Save.

When a subsequent measurement meets the condition (turbidity > 5.0), the system will:

- Create an **AlertEvent**.
- Attempt to send an email to the specified recipient.

Note: For email to work, email settings must be correctly set in .env (see next section).

---

6. Email Settings

For alert rules and sensor health alerts to send email, the following variables need to be set in .env:

env

```
DEFAULT_FROM_EMAIL=smartenv@kommun.se
EMAIL_HOST=smtp.kommun.se
EMAIL_PORT=587
EMAIL_USE_TLS=True
EMAIL_HOST_USER=smartenv@kommun.se
EMAIL_HOST_PASSWORD=<password_or_app_password>

SENSOR_HEALTH_EMAIL_TO=drift@kommun.se
```

Recommendation:

- Use a dedicated account (e.g., smartenv@kommun.se) with an app password.
- Then test with:

bash

```
docker compose exec web python manage.py check_sensor_health --stale-minutes 0 --
email-to drift@kommun.se
```

This forces a test alert (all sensors are considered stale with a threshold of 0 minutes).

---

7. Monitoring and Operations

7.1. Health endpoint

Endpoint:

- GET /health/

Response:

- 200 OK with JSON status (e.g., {"status": "ok", "database": "connected"}) when everything works.
- 503 Service Unavailable if the backend cannot reach the database.

Usage:

- Can be connected to the municipality's monitoring system (e.g., ping every 30 seconds).

7.2. Sensor health (CLI command)

Command:

bash
```
docker compose exec web python manage.py check_sensor_health --stale-minutes 30 --email-to drift@kommun.se
```

Function:

- Counts the number of sensors.
- Marks which are OK and which are Stale.
- Sends an email if any sensor is stale (if --email-to or SENSOR_HEALTH_EMAIL_TO is set).
- Can be connected to a scheduler (cron, Windows Task Scheduler, etc.).

---

8. API Overview (for technicians)

The most important API endpoints:

- **Ingest (sensor → backend)**
  - POST /api/data/
  - Auth: X-Device-Key: <DEVICE_API_KEY> or Authorization: Bearer <DEVICE_API_KEY>
  - Rate limiting: sensor_ingest (e.g., 120 requests per minute).
- **Latest value**
  - GET /api/data/latest/?sensor_id=<id>
  - Auth: requires logged-in user (only for internal systems/users).
  - Rate limiting: sensor_read.
- **History**
  - GET /api/data/history/?sensor_id=<id>&from=YYYY-MM-DD&to=YYYY-MM-DD
  - Auth: logged-in user.
- **Incident export**
  - GET /api/incidents/
  - Auth: requires Manager or Admin.

---

9. Common Errors and Solutions

9.1. Dashboard shows 502 Bad Gateway

Cause:

- The proxy (Caddy) is running, but the web container has crashed (often due to a wrong DATABASE_URL).

Solution:

1. Check the logs:

bash
```
docker compose logs web
```

2. Verify that .env contains:

env
```
DATABASE_URL=postgresql://smartenv:smartenv@db:5432/smart_env_monitor
```

3. Restart:

bash
```
docker compose up -d --build
```

9.2. Simulator shows “Connection Error”

Cause:

- The backend is not responding (web/proxy down) or the BACKEND_URL is incorrect.

Solution:

- Verify that the Docker stack is running (docker compose ps).
- Verify that .env contains:

env

```
BACKEND_URL=http://localhost:18080/api/data/
```

9.3. Simulator shows “Unauthorized”

Cause:

- The DEVICE_API_KEY in the simulator does not match the DEVICE_API_KEY in the backend.

Solution:

- Set the same value in .env:

env

DEVICE_API_KEY=<same_key_in_backend_and_simulator>

9.4. /api/data/ shows “Method GET not allowed”

Cause:

- You opened the endpoint in a browser. It is designed for POST (sensor ingest), not for GET.

Solution:

- This is expected behaviour. Use the dashboard (/dashboard/) or the read API (/api/data/latest/, /api/data/history/) to view data.

---

10. Summary

- Start the system: .env → docker compose up -d --build.
- Create admin: docker compose exec web python manage.py createsuperuser.
- Start simulation: python sensor/simulator.py.
- Monitor:
  - Dashboard: http://localhost:18080/dashboard/
  - Health: http://localhost:18080/health/
  - Sensor health: docker compose exec web python manage.py check_sensor_health.
- Alerts and rules:
  - Create alert rules in /admin/api/alertrule/.
  - View alert history at the bottom of the dashboard and in /admin/api/alertevent/.